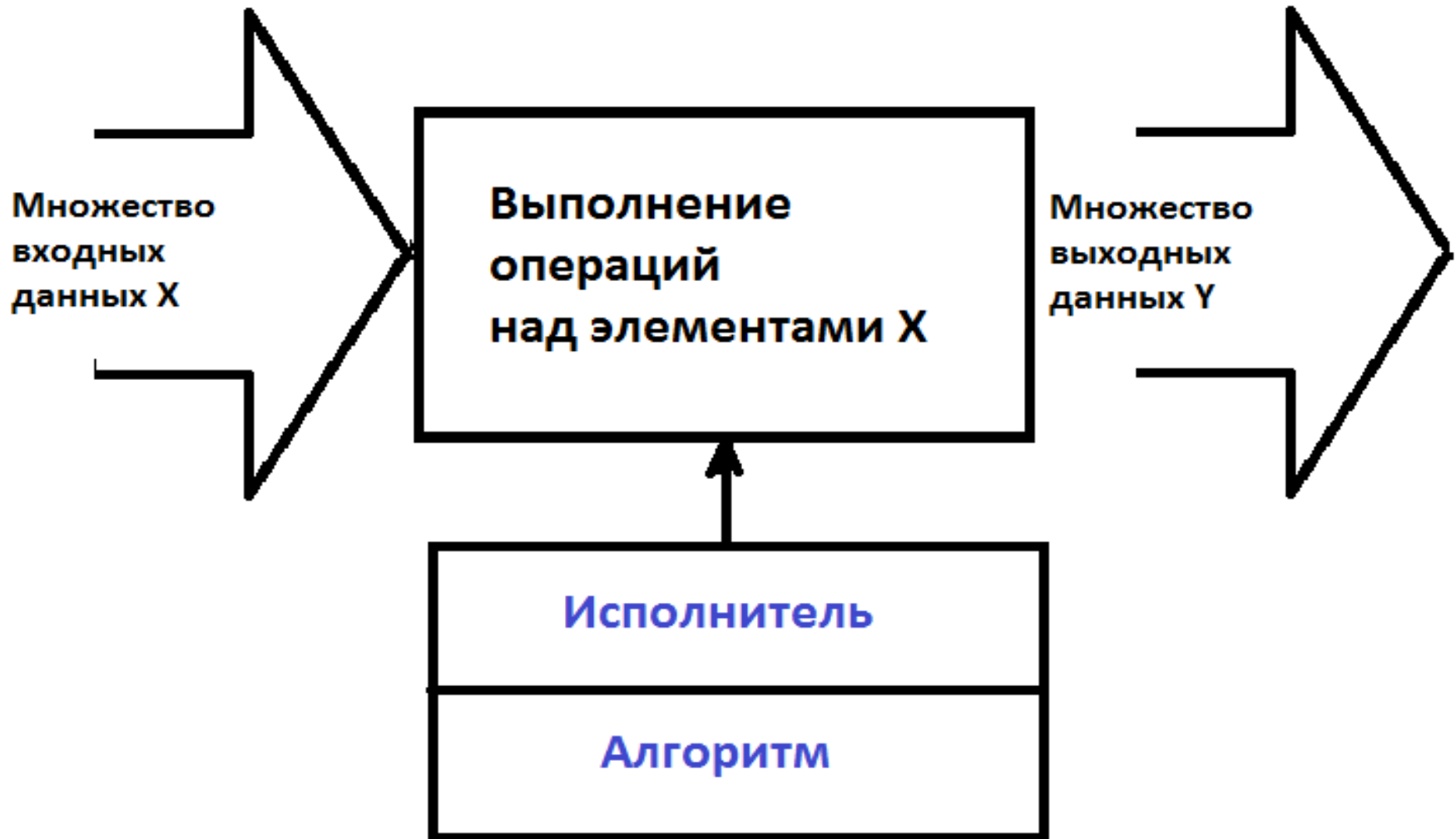


Разработка клиент-серверных приложений

Лекция 1

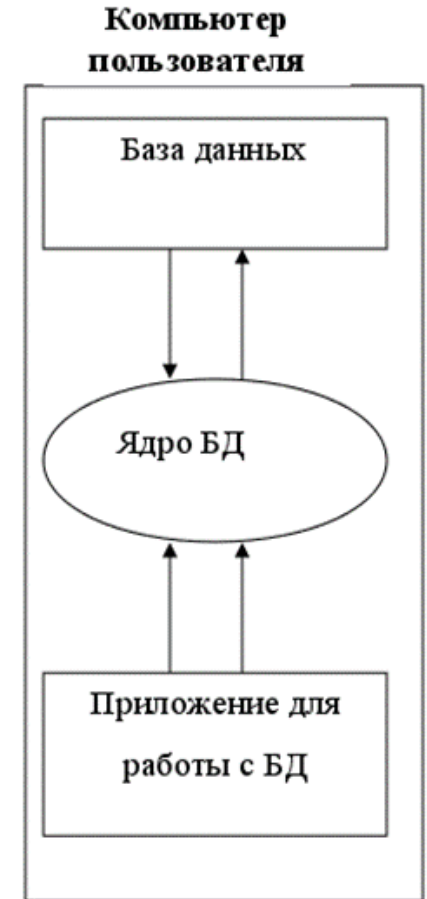
Введение в технологию клиент-сервер.
Введение в язык разметки HTML.

Обработка данных



Локальная архитектура

- При работе с локальными базами данных (БД) сами БД расположены на том же компьютере, что и приложения, осуществляющие доступ к ним.
- Работа с БД происходит в однопользовательском режиме.
- Ядро БД расположено на компьютере пользователя.
- Приложение ответственно за поддержание целостности БД и за выполнение запросов к БД.



Клиент-серверная архитектура. Основные роли

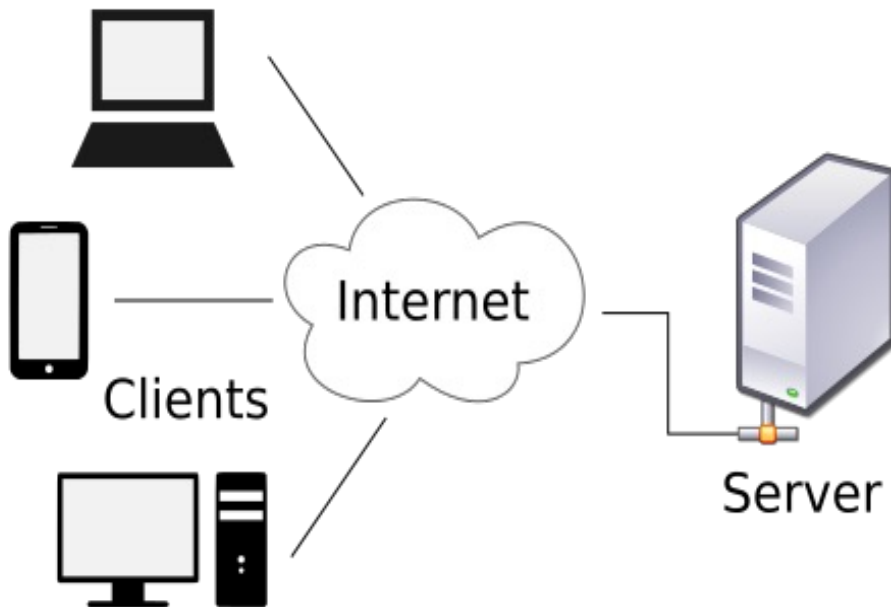
Сервер – компьютер, управляющий тем или иным ресурсом.

Клиент – компьютер (или программа), запрашивающий и пользующийся каким-либо ресурсом.

Важно: для одного ресурса можно выполнять роль клиента, для другого – сервера. Пример: сервер приложений (сервер для удаленного пользователя и клиент для сервера БД).

Основные компоненты архитектуры «клиент-сервер»

Архитектура клиент-сервер (*client-server architecture*) — вычислительная или сетевая архитектура, в которой задания или сетевая нагрузка распределены между поставщиками услуг, называемыми серверами, и заказчиками услуг, называемыми клиентами.



Основные компоненты:

- Сервер (или набор серверов);
- клиент (или набор клиентов);
- сеть передачи данных.

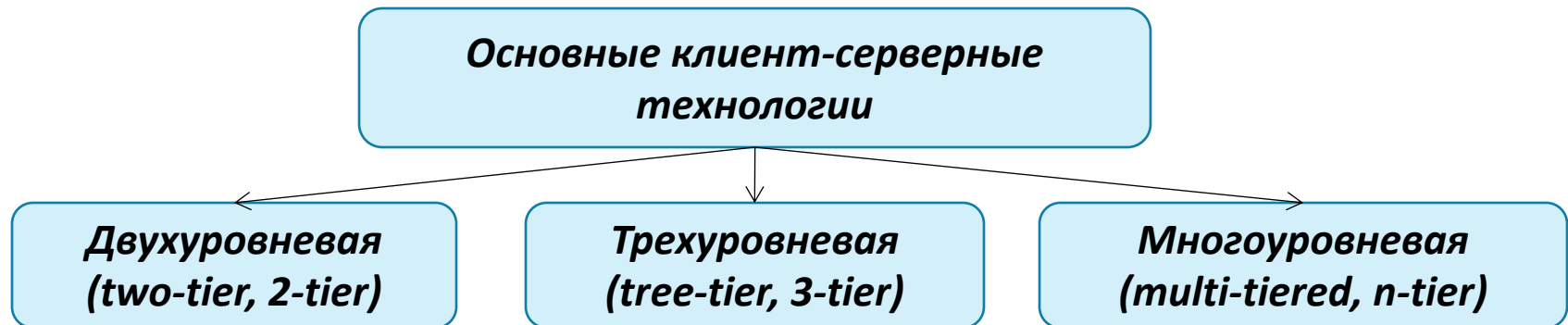
Основные уровни:

- уровень представления;
- прикладной уровень;
- уровень управления ресурсами или данными.

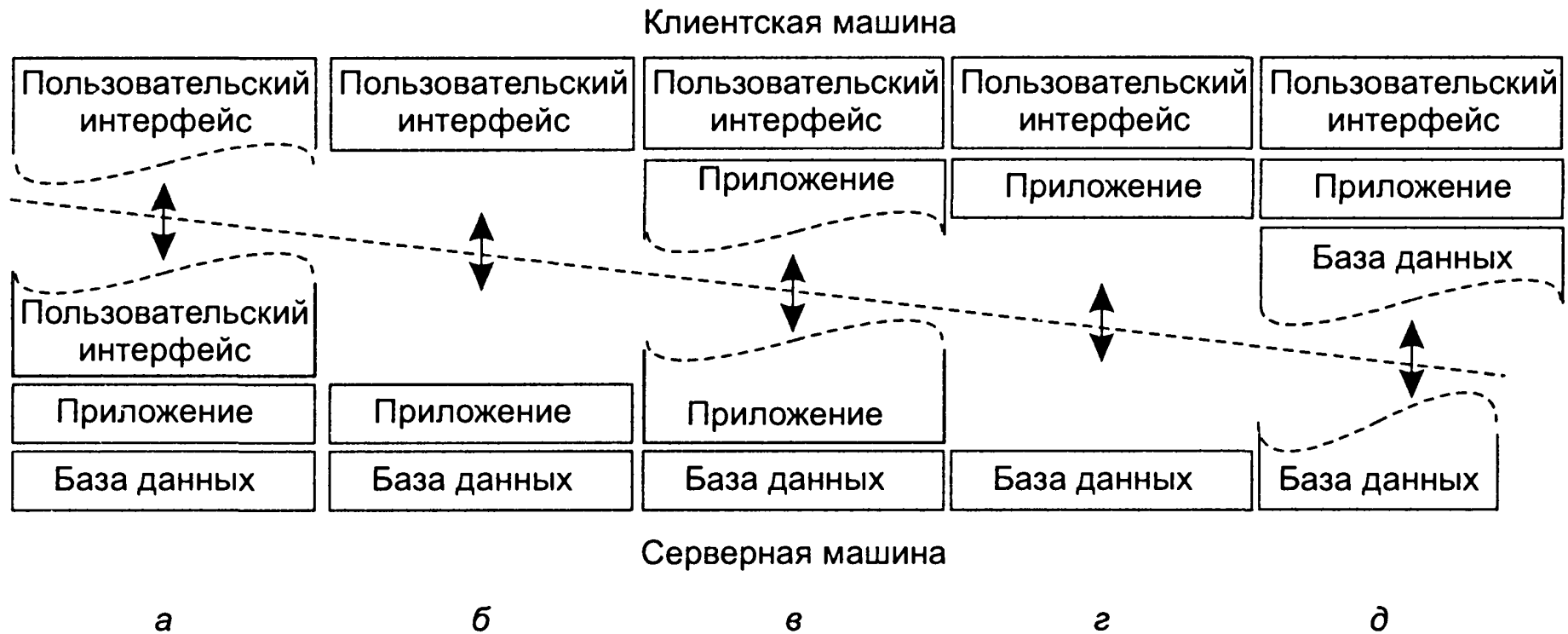
Клиент-серверные технологии

Клиент-серверная технология – практическая реализация архитектуры «клиент-сервер».

Протокол обмена (взаимодействия) – правила, по которым осуществляется связь между компонентами клиент-серверной технологии.



Альтернативные формы организации архитектуры клиент-сервер



Виды клиентов



Тонкий клиент, thin client

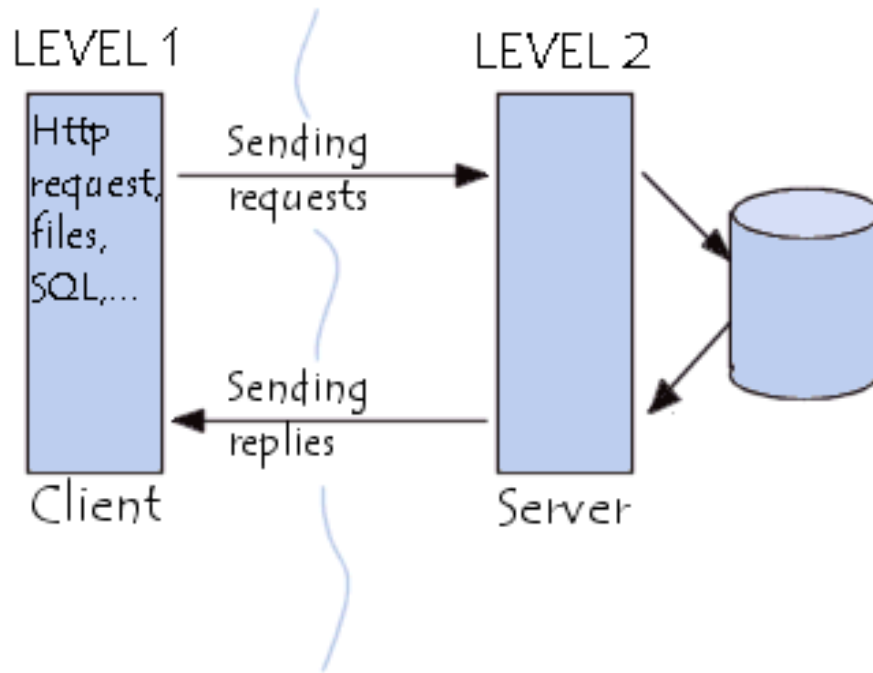
В компьютерных технологиях — компьютер или программа-клиент в сетях с клиент-серверной или терминальной архитектурой, где большая часть задач по обработке информации перенесена на сервер и права доступа клиента строго ограничены. Примером тонкого клиента может служить компьютер с браузером, использующийся для работы с веб-приложениями.

Толстый клиент, rich client

В архитектуре клиент-сервер — это приложение, обеспечивающее (в противовес тонкому клиенту) полную функциональность и независимость от центрального сервера.

Часто сервер в этом случае является лишь хранилищем данных, а вся работа по обработке и представлению этих данных переносится на машину клиента.

Двухуровневая архитектура



Недостатки

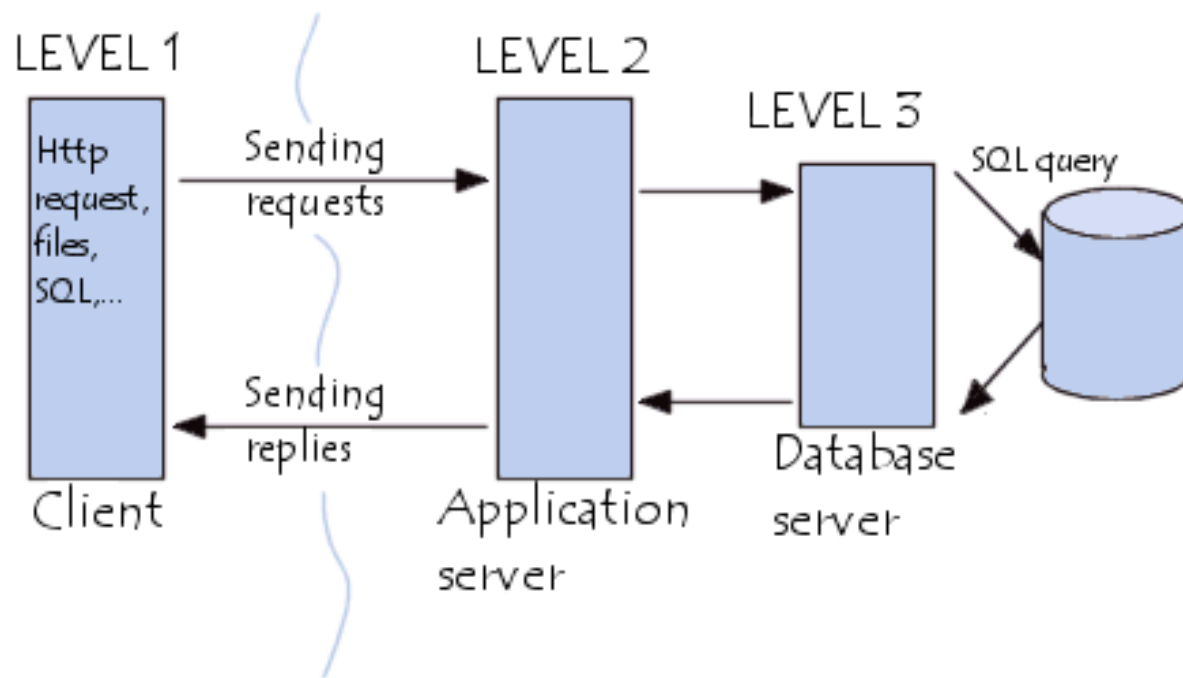
- «толстый» клиент:

- сложность администрирования;
- усложняется обновление ПО;
- усложняется распределение полномочий;
- перегрузка сети из-за передачи необработанных данных;
- слабая защита данных.

- «тонкий» клиент:

- усложняется реализация;
- низкая производительность и надежность программ;
- вероятность выхода из строя всего сервера БД из-за ошибки в программе;
- программы полностью непереносимы на другие системы и платформы.

Трехуровневая архитектура



Преимущества

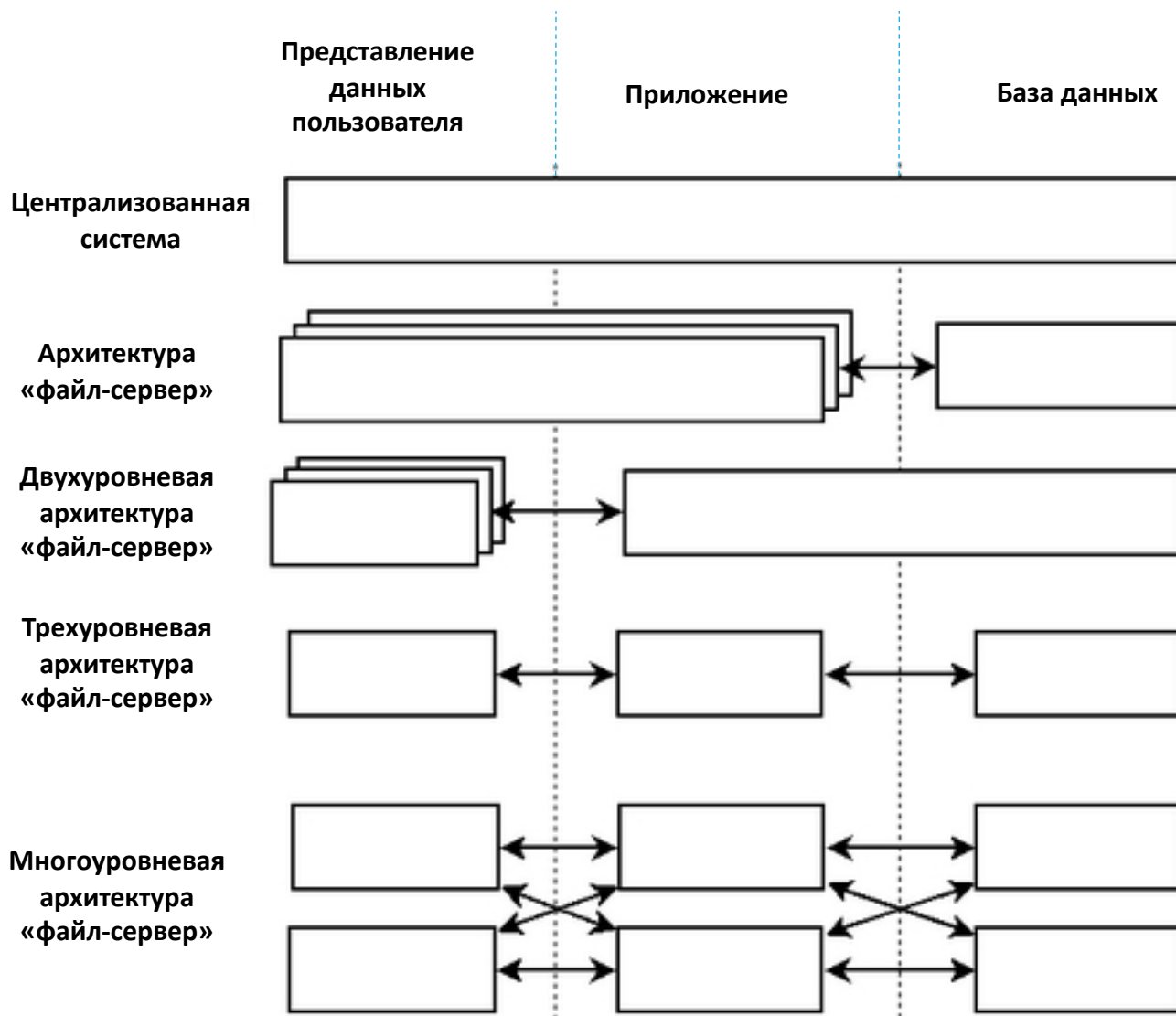
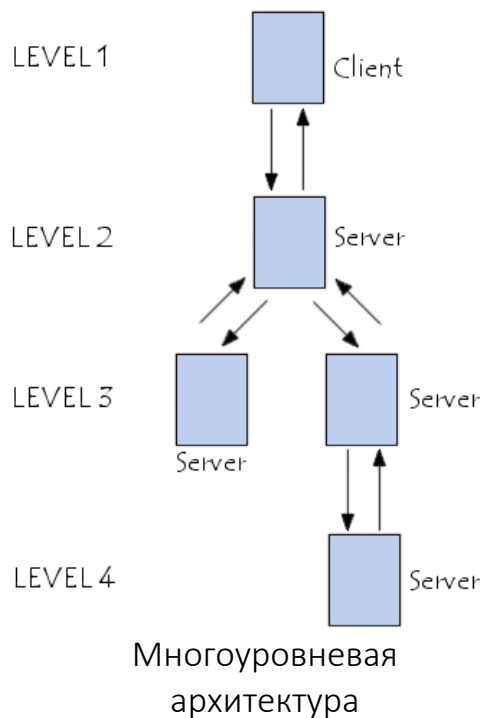
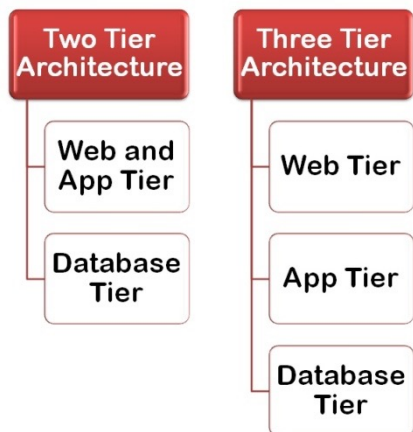
Высокий уровень:

- гибкости;
- масштабируемости;
- безопасности;
- производительности.

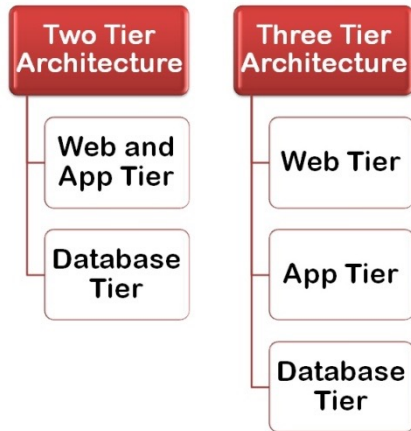
Архитектура разделена между:

- *клиентом* (запрашивает ресурсы, имеет GUI);
- *сервером приложений* (middleware) – обеспечивает требуемые ресурсы через другой сервер;
- *сервером данных* (обеспечивает сервер приложений нужными данными).

Сравнение архитектур



Сравнение архитектур. Достоинства и недостатки

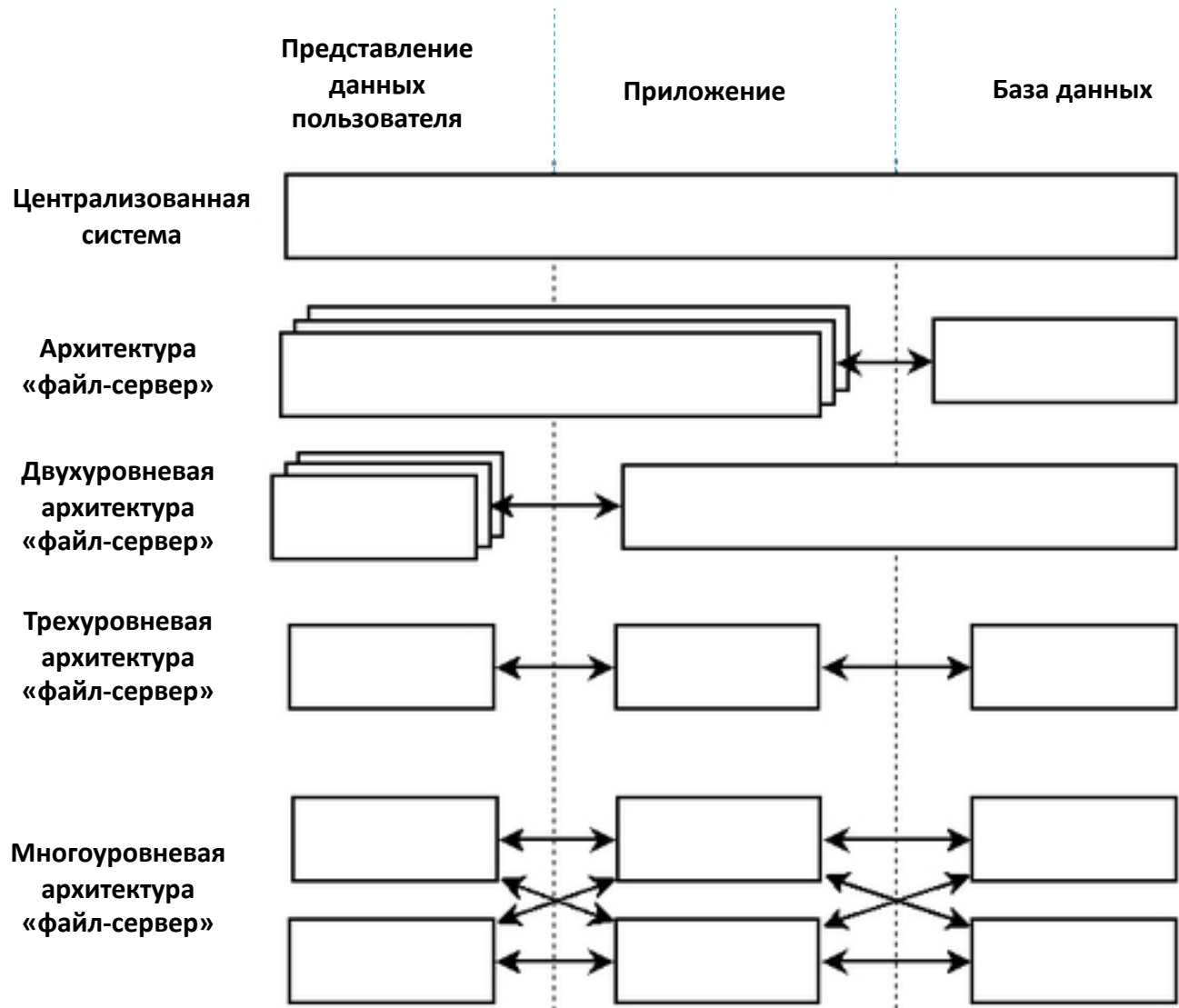


Достоинства:

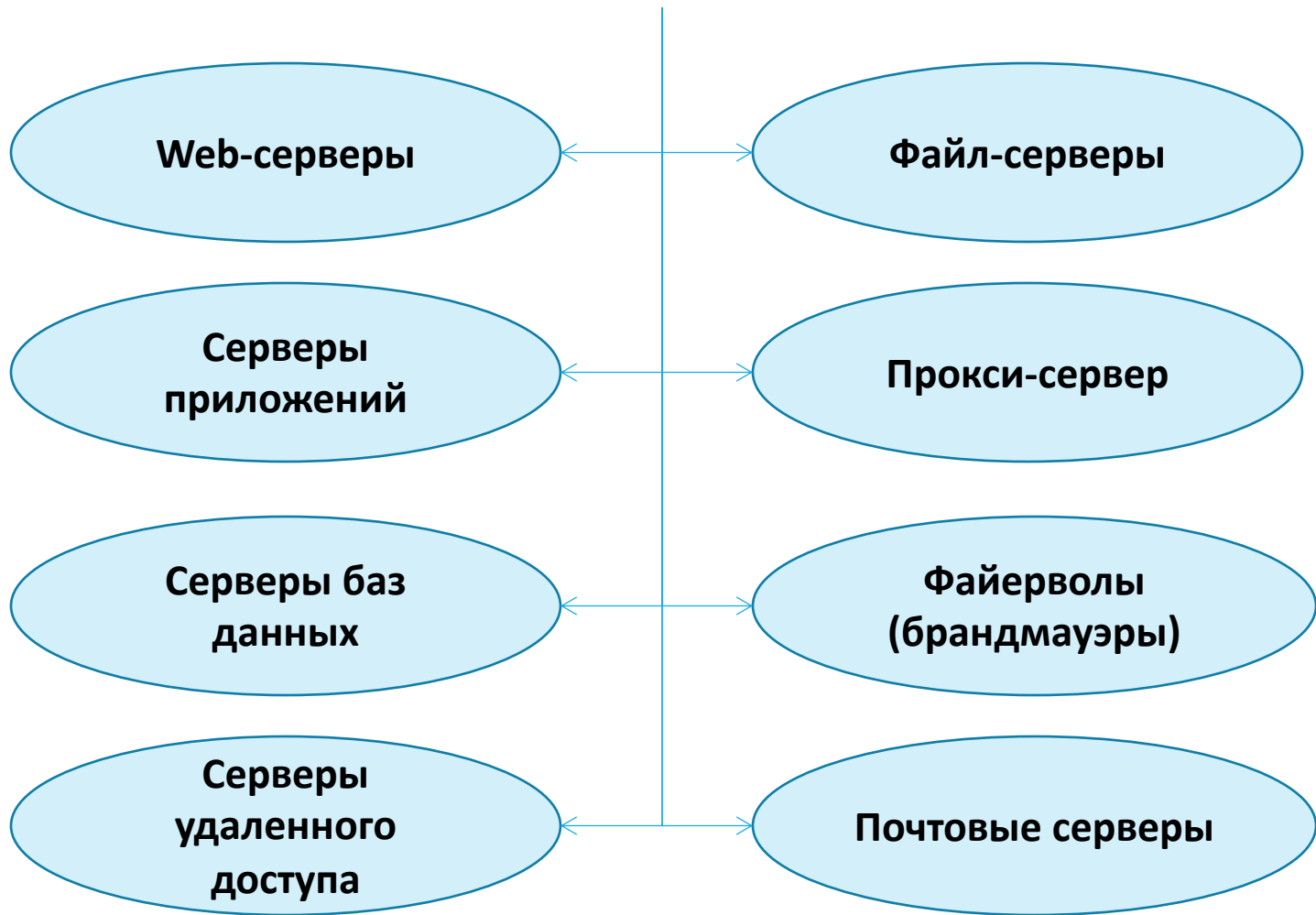
- централизованные ресурсы;
- улучшенная безопасность;
- администрирование на уровне сервера;
- масштабируемая сеть.

Недостатки:

- увеличение стоимости;
- «слабость» сервера;
- проблема трафика.

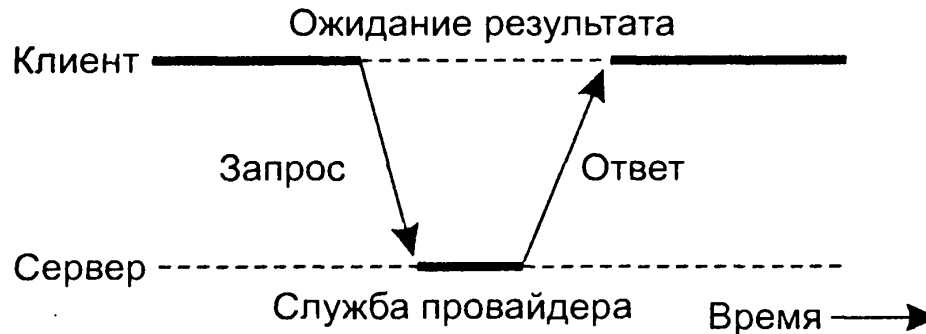


Разновидности серверов

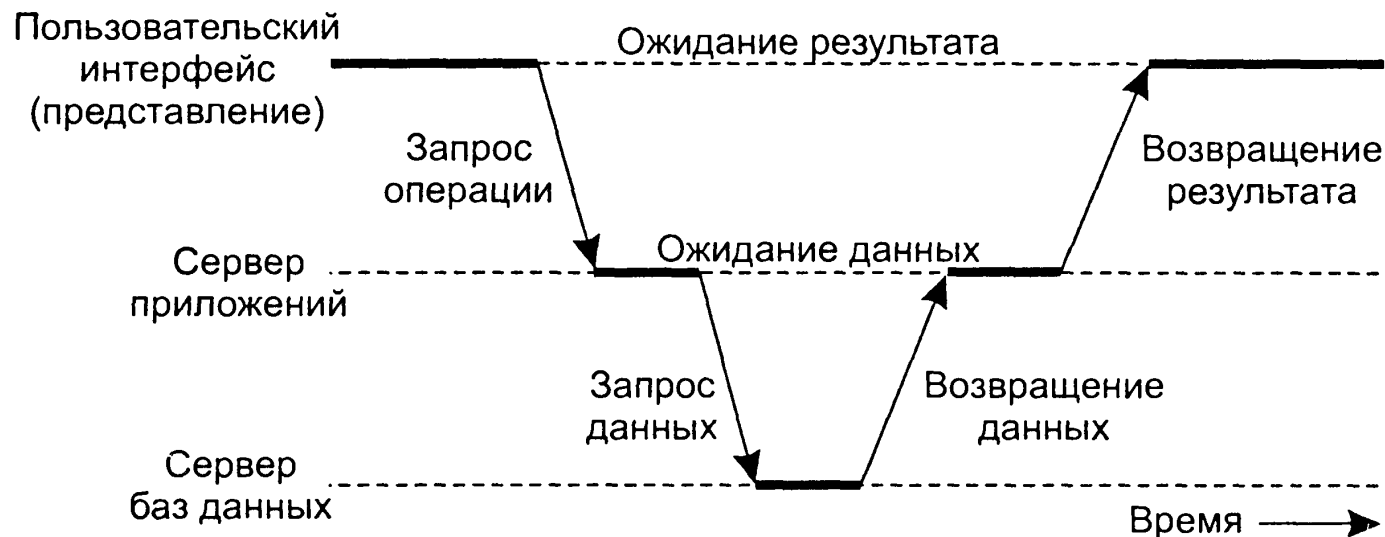


Взаимодействие между клиентом и сервером во времени

Двухуровневая архитектура



Трёхуровневая архитектура



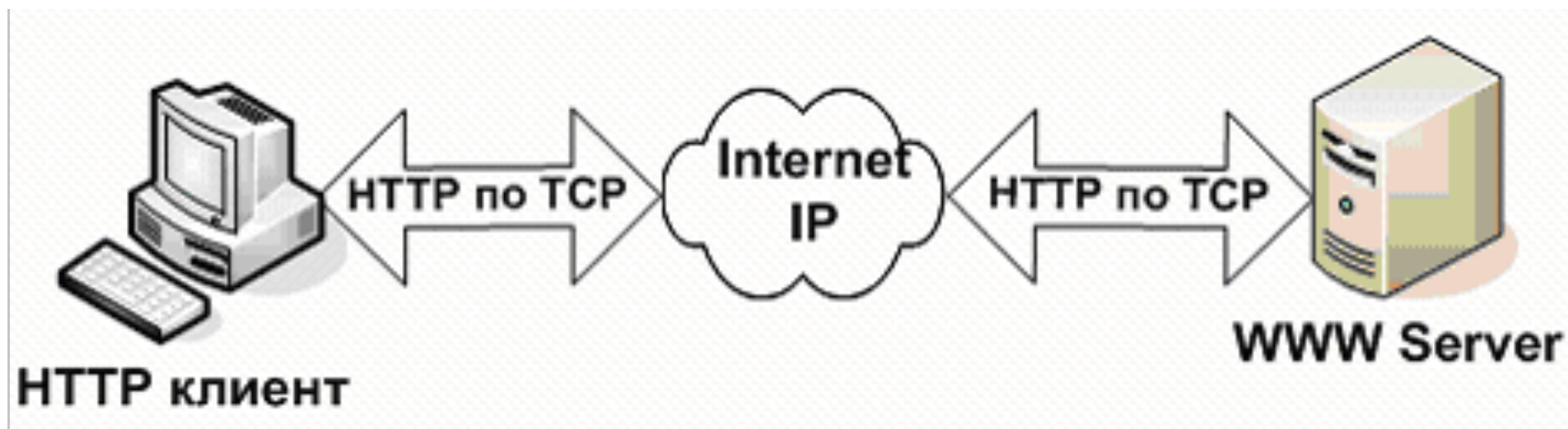
Веб-сервер

Программа, запущенная на узле сети Интернет и выдающая посетителям этого узла web-страницы по запросам. Также web-сервером называется узел, на котором запущена WWW-служба, или даже компьютер, являющийся таким узлом.

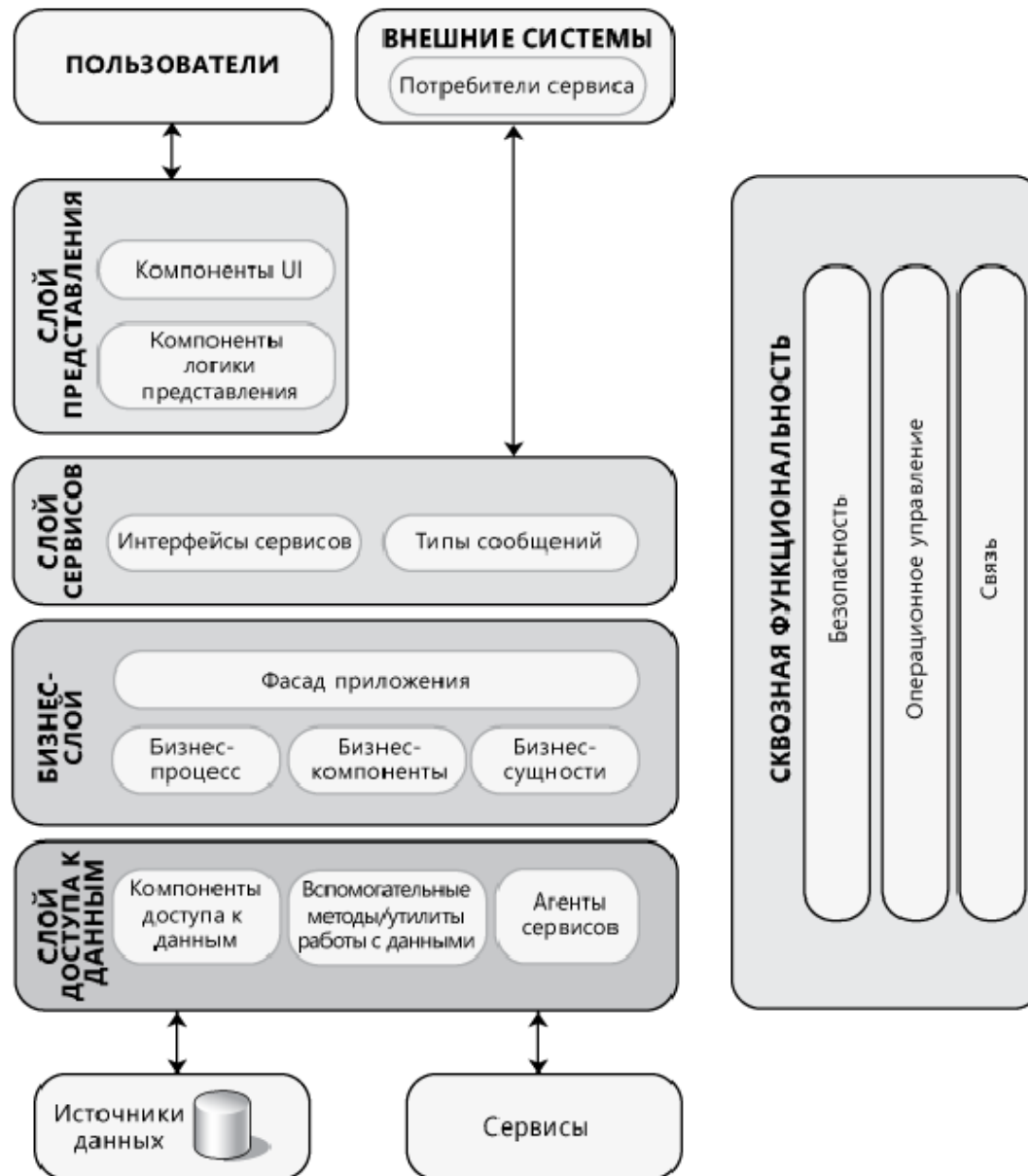
WWW - основная служба в сети Интернет, позволяющая получать доступ к информации на любых веб-серверах, подключенных к сети.

WWW построена по схеме "клиент-сервер".

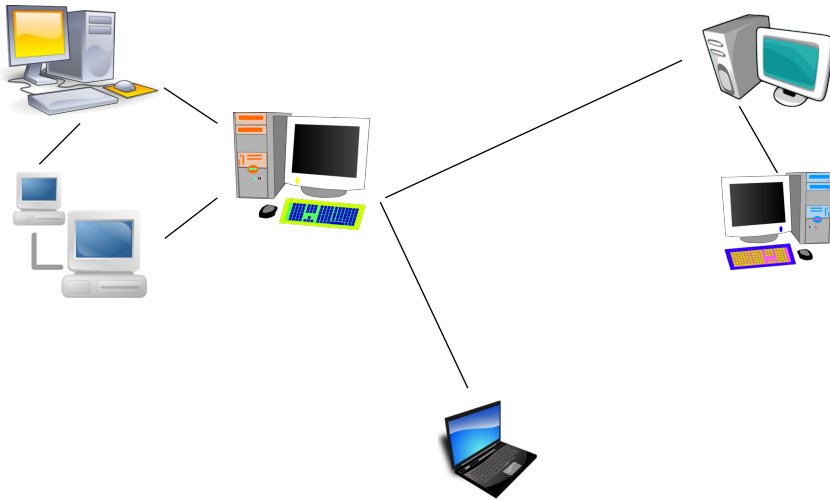
В качестве клиента выступает браузер, который является также и интерпретатором HTML.



Пример архитектуры приложения



Рождение интернета – 1969 г.

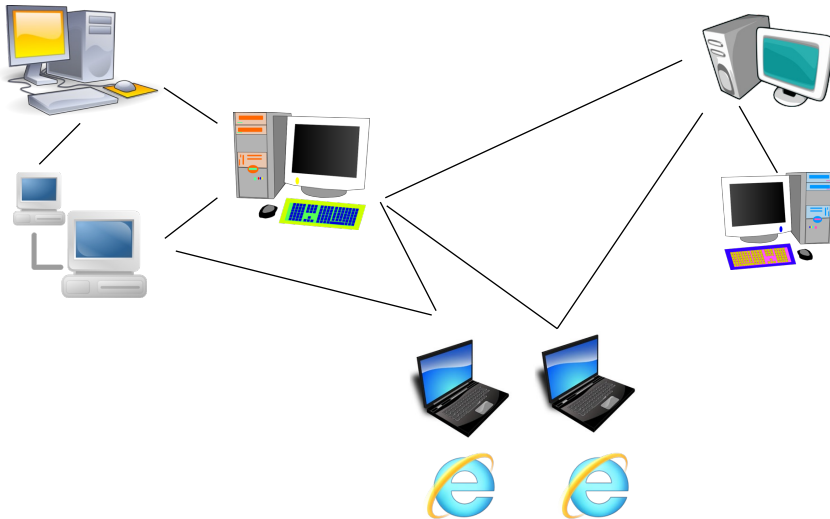


FTP – передача файлов

E-mail – электронная почта

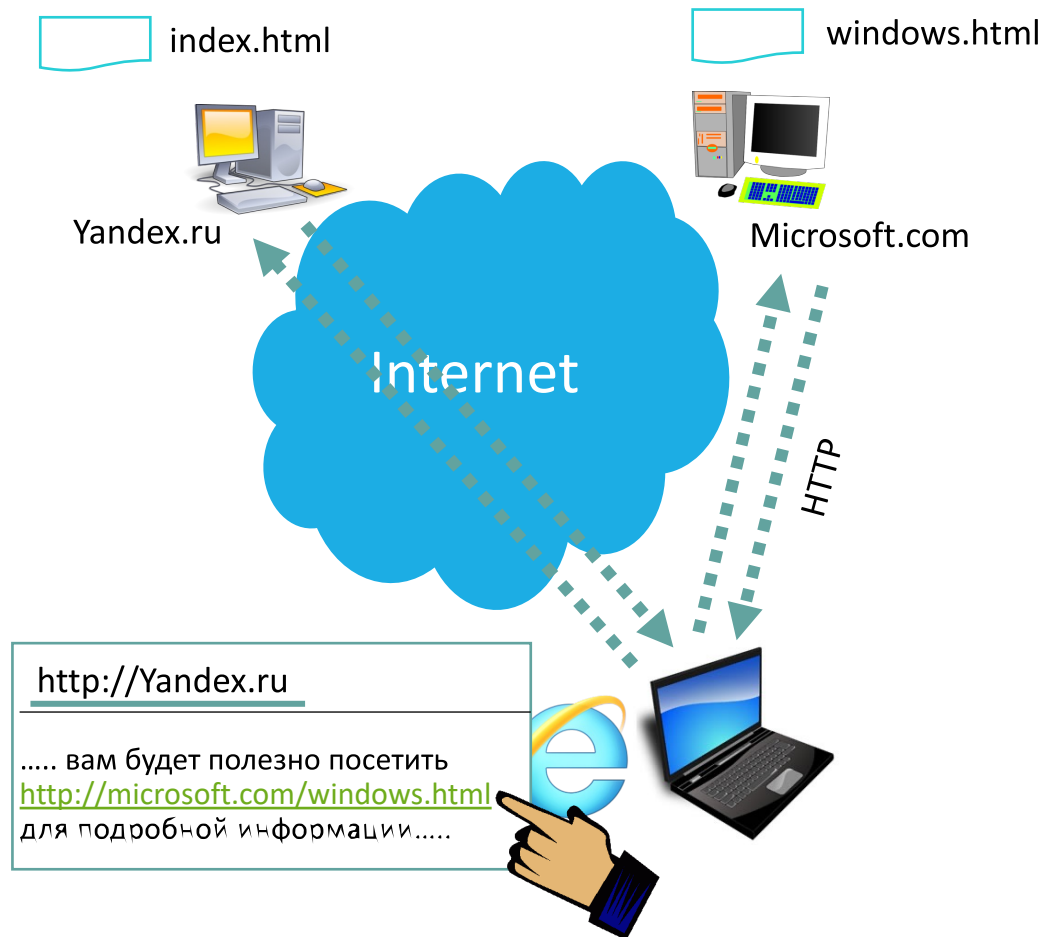
telnet – удалённый сеанс

Всемирная паутина – 1990 г.

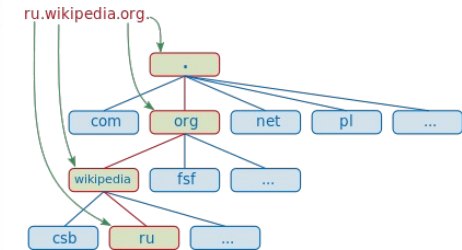
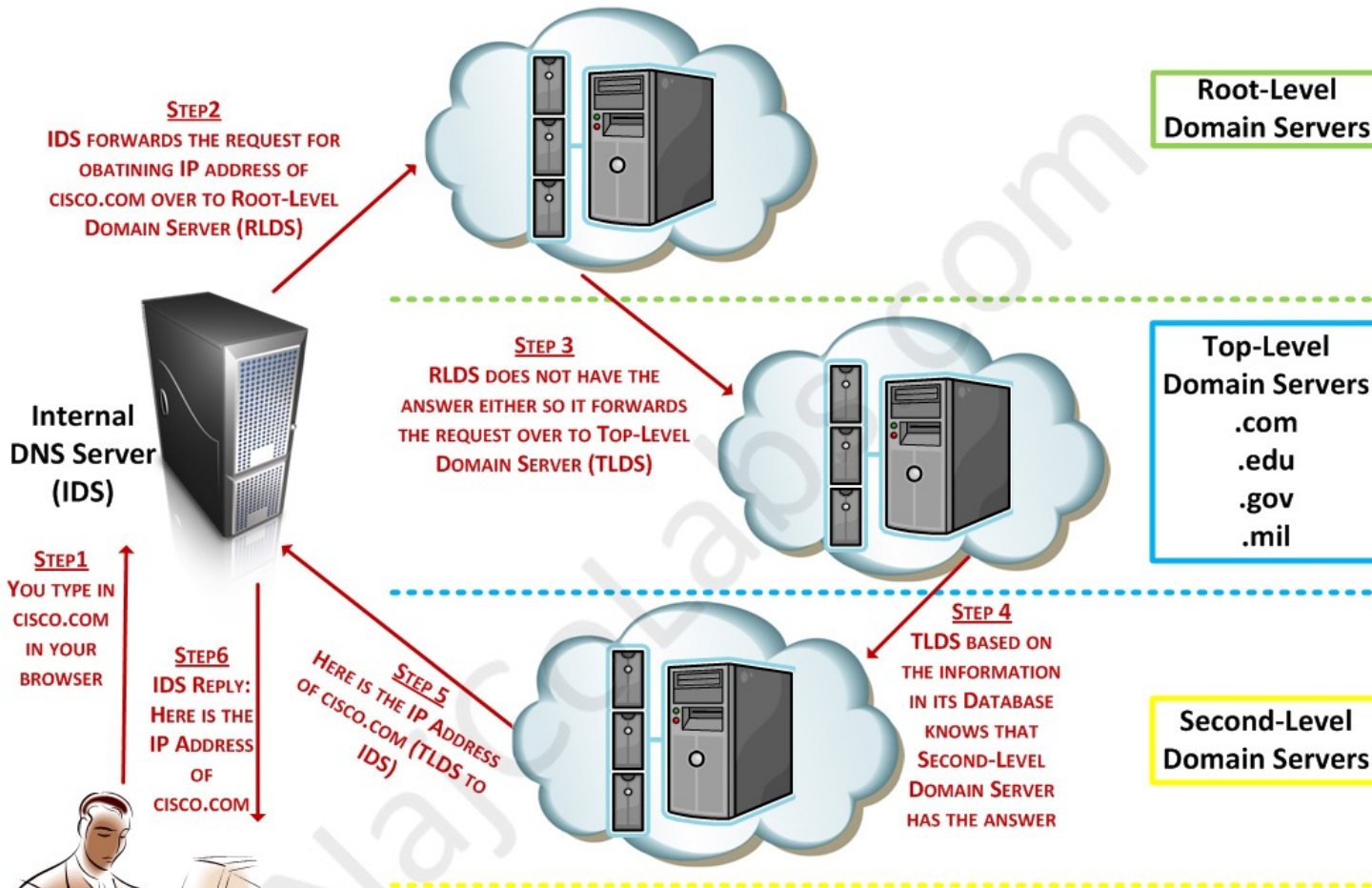


- Единый формат документов – HTML
- Браузер для форматирования
- Гипертекст: связи между узлами в виде ссылок

Как устроена паутина

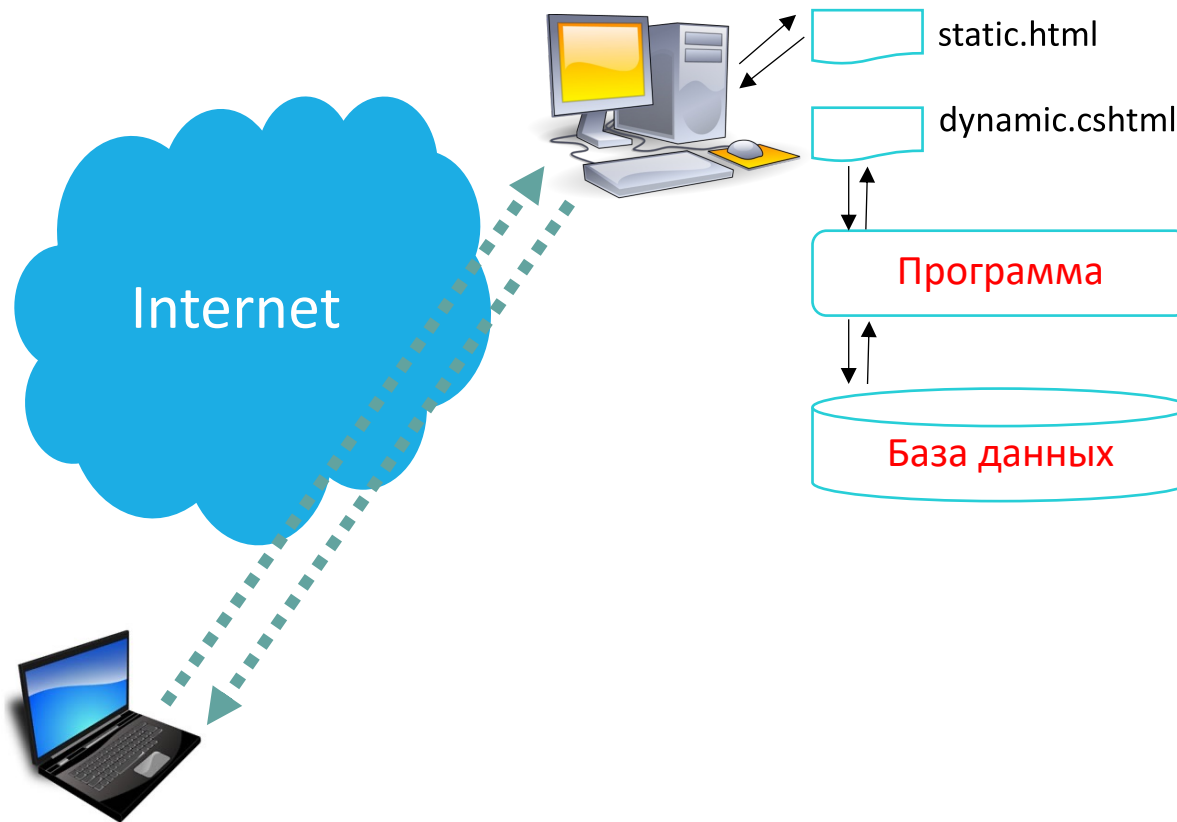


DNS: Domain Name System

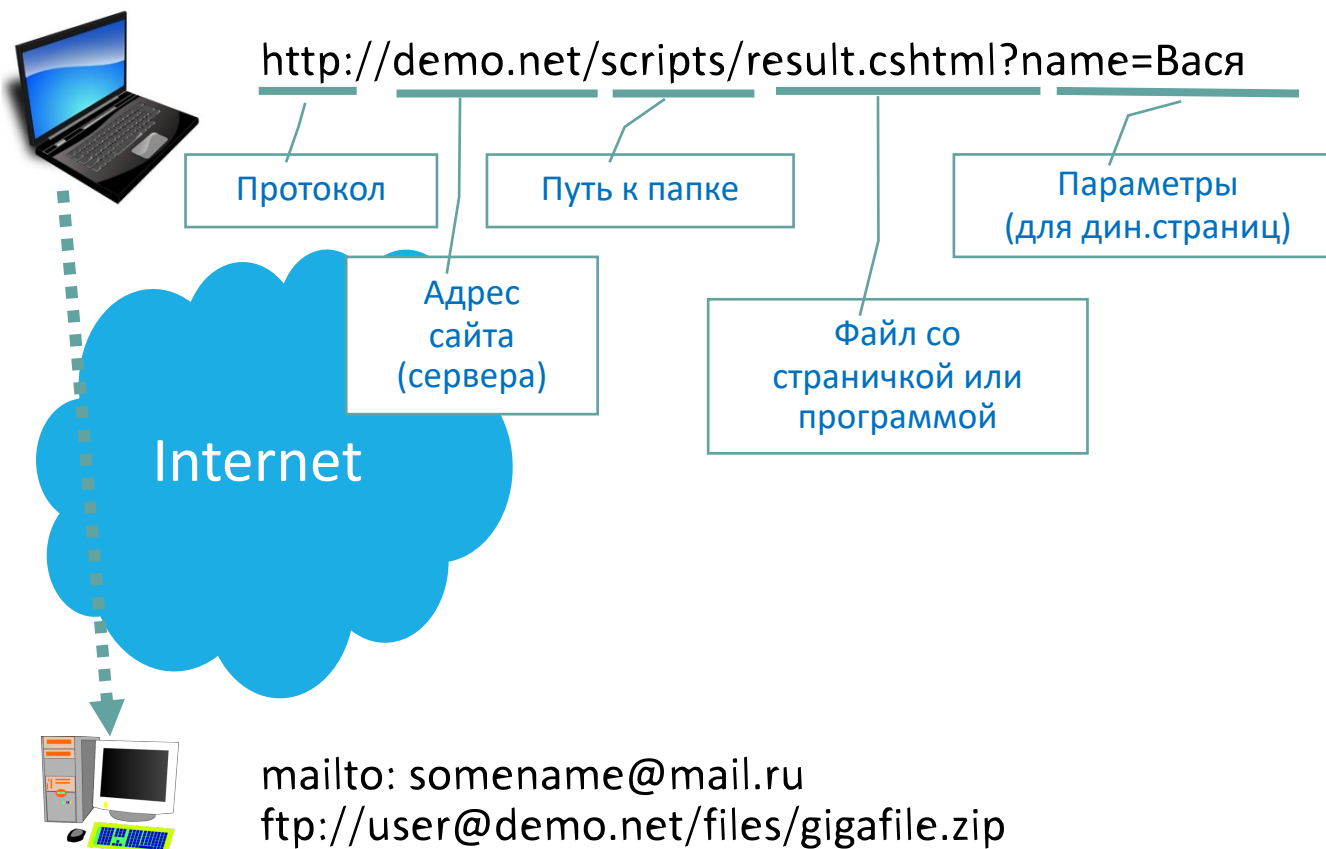


How DNS Works?

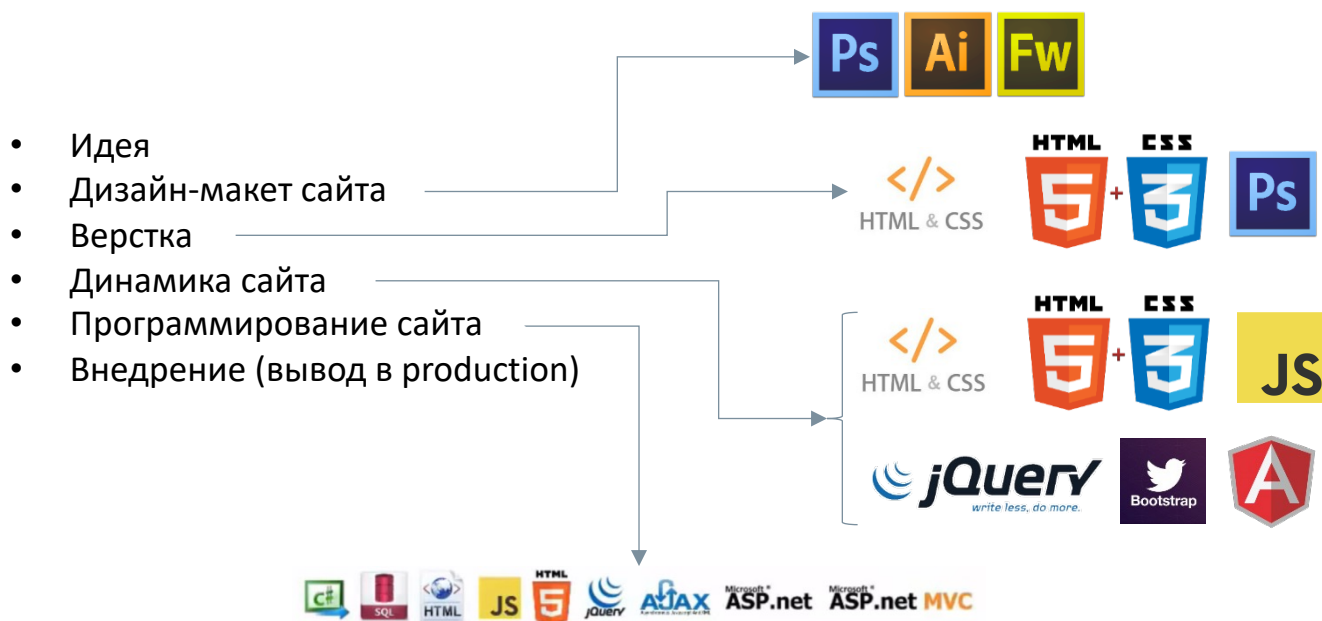
Статические vs. динамические страницы



URL – Uniform Resource Locator



Этапы создания Web-сайта



HTML

Hyper Text Markup Language (HTML, язык гипертекстовой разметки) — стандартизированный язык разметки документов во Всемирной паутине.

Я **очень** люблю:

- компьютеры
- кошек (*иногда*)

Я **очень** люблю:

компьютеры

кошек (иногда)



Проверить соответствие стандартам W3C:

<http://validator.w3.org>

Зачем так сложно?

Различное форматирование для различных устройств:

```
Я <strong>очень</strong> люблю:  
<ul>  
  <li>компьютеры</li>  
  <li>кошек (<em>иногда</em>)</li>  
</ul>
```



Я **очень** люблю:

- компьютеры
- кошек (*иногда*)



Я **ОЧЕНЬ** люблю:

- * компьютеры
- * кошек (иногда)

Некоторые термины

- **Элемент** (element) — конструкция языка HTML. Это контейнер, содержащий данные и позволяющий отформатировать их определенным образом.
- **Тег (tag)** — начальный или конечный маркеры элемента. Теги определяют границы действия элементов и отделяют элементы друг от друга. В тексте Web-страницы теги заключаются в угловые скобки, а конечный тег всегда снабжается косой чертой.
- **Атрибут** (attribute) — параметр или свойство элемента. Это переменная, которая имеет стандартное имя и которой может присваиваться определенный набор значений. Атрибуты располагаются внутри начального тега и отделяются друг от друга пробелами.
- **Гиперссылка** — фрагмент текста, который является указателем на другой документ или файл/объект. Гиперссылки необходимы для того, чтобы обеспечить возможность перехода от одного документа к другому.
- **Web-страница** — документ (файл), подготовленный в формате гипертекста и размещенный в World Wide Web.
- **Сайт** (site) — набор Web-страниц, принадлежащих одному домену.

HTML: основные понятия

Я очень люблю:

 кот
 кошек <i>иногда</i>

</br>

Открывающий тег

Закрывающий тег

Одиночный тег

Тег

HTML: структура страницы

```
<!DOCTYPE html>
```

```
<html>
```

```
  <head>
```

```
    <title>My Book Catalog</title>
```

```
  </head>
```

```
  <body>
```

```
    ...
```

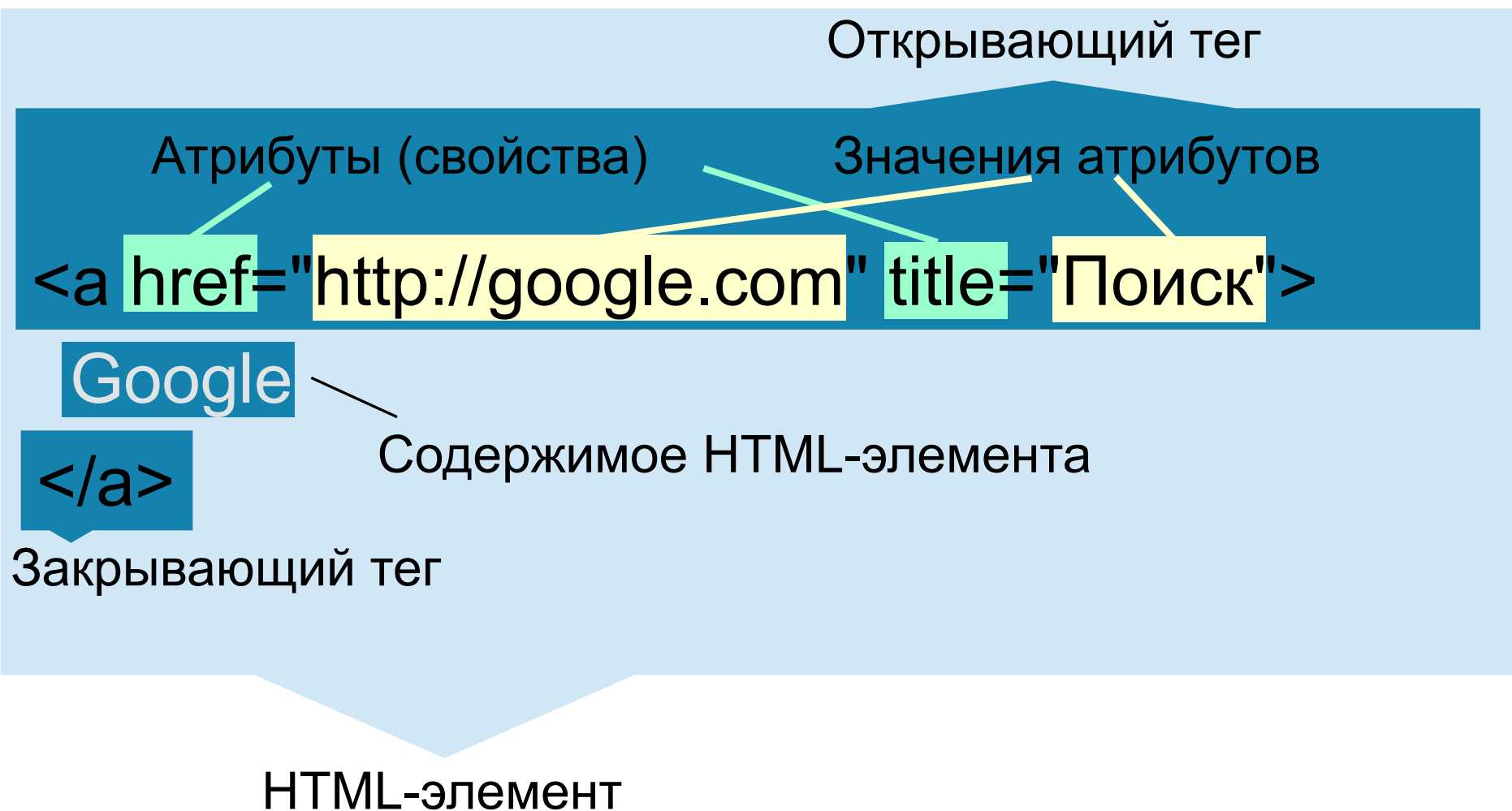
```
  </body>
```

```
</html>
```

Заголовок
(описание)

Тело страницы

HTML-элемент с содержимым



Вложенность парных тегов

Правильное вложение:



```
<div><p>Lorem <b>ipsum dolor</b> sit amet...</p></div>
```

Неправильное вложение:



```
<div><p>Lorem <b>ipsum <i>dolor</b> sit</i> amet...</div></p>
```

Заголовки и текст

Заголовки по убыванию важности:

<h1></h1>
<h2></h2>
<h3></h3>
<h4></h4>
<h5></h5>
<h6></h6>

Абзац: **<p></p>**

<h1>Название статьи</h1>
<h2>Глава 1</h2>
<p>Абзац</p>
<p>Абзац</p>
<h2>Глава 2</h2>
<h3>Глава 2.1</h3>
<p>Абзац</p>
<h3>Глава 2.2</h3>
<p>Абзац</p>
<h2>Глава 3</h2>
<p>Абзац</p>

Название статьи

Глава 1

Абзац

Абзац

Глава 2

Глава 2.1

Абзац

Глава 2.2

Абзац

Глава 3

Абзац



Ссылки

`<a href="http://google.com"` ← Адрес
`title="Google"` ← Всплывающая подсказка
`target="_blank">` ← Открытие в той же или в новой вкладке

Go to Google

`</a>`

Ссылкой может быть:

- Просто текст
- Абзац (один или несколько)
- Заголовок (один или несколько)
- Изображение
- И проч.

Изображения

`` ← Поясняющий текст *

* — Обязательные атрибуты

Универсальные контейнеры и атрибуты

Универсальные контейнеры

`<div>Содержимое</div>` ← Блочный контейнер

`<span>Содержимое</span>` ← Строчный контейнер

Нужны для формирования общей структуры документа, компоновки элементов, выделения подэлементов (например, часть ссылки или параграфа).

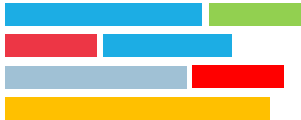
Универсальные атрибуты

`id = "header-logo"` ← Идентификатор

`class = "block-title"` ← Класс

Нужны для обращения к элементам из CSS или JS. Назначаемый идентификатор должен быть уникальным в контексте одного HTML-документа. Несколько элементов может иметь один и тот же класс. Каждый элемент может иметь несколько классов (значения указываются в кавычках через пробел).

Блочные и строчные элементы



1. Строчные элементы в общем потоке располагаются последовательно на одной строке один за другим. К строчным элементам относятся теги `<img>`, `<span>`, `<a>`, `<q>`, `<code>` и др., а также элементы, у которых свойство **display** установлено как **inline**.
2. Блочные элементы в общем потоке располагаются последовательно один под другим. По умолчанию два блочных элемента не могут располагаться на одной строке. К блочным элементам относятся теги `<address>`, `<blockquote>`, `<div>`, `<fieldset>`, `<form>`, `<h1>`, ..., `<h6>`, `<hr>`, `<ol>`, `<p>`, `<pre>`, `<table>`, `<ul>` и некоторые устаревшие. Также блочным становится элемент, если в стиле для него свойство **display** задано как **block**, **list-item** или **table**.

Также существуют строчно-блочные элементы, которые заимствуют свойства от обоих классов элементов. С помощью CSS можно менять тип элементов.

Элементы для работы с текстом

<code><p> </p></code>	Тег для создания блока текста — параграфа.
<code><pre></pre></code>	Контейнер, отображающий содержимое с учетом форматирования.
<code>
</code>	Непарный тег для переноса текста на следующую строку.
<code>; </code>	Отображает содержимое с полужирным начертанием.
<code><i></i>; </code>	Отображает содержимое с курсивным начертанием.
<code><sub></sub></code>	Отображает содержимое в нижнем индексе.
<code><sup></sup></code>	Отображает содержимое в верхнем индексе.
<code></code>	Предназначен для форматирования текста документа.
<code><center></center></code>	Выравнивает содержимое по центру относительно родительского элемента.

Важная орг. информация

Контроль успеваемости

- Оценка текущего контроля - результаты работы на семинаре
(лабораторные работы);
- Курсовая работа (принимается семинаристом):
 - клиент-серверное приложение по утвержденной тематике;
 - пояснительная записка;
 - презентация.
- Экзамен.